



Polymorphism

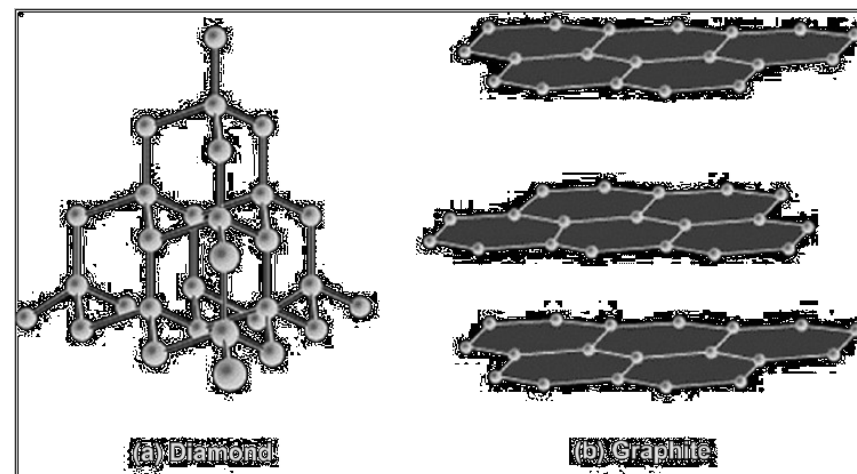
Poly= Multi ; morphe= Form

Allows us to perform a single action in different ways.

Recap organic chemistry

Carbon has this property

The crystal structures are different
Diamond vs Graphite



Polymorphism

Let's go back to the Library example

Issue book ()

Can be used by both Students

Can be used by Faculties

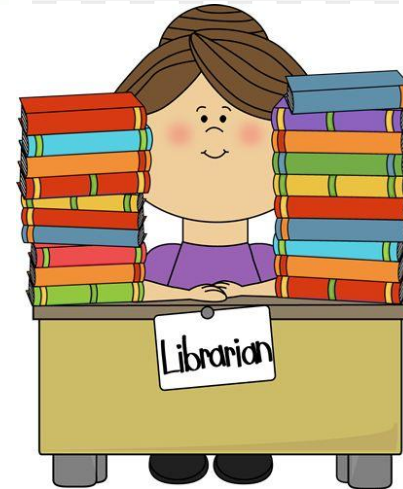
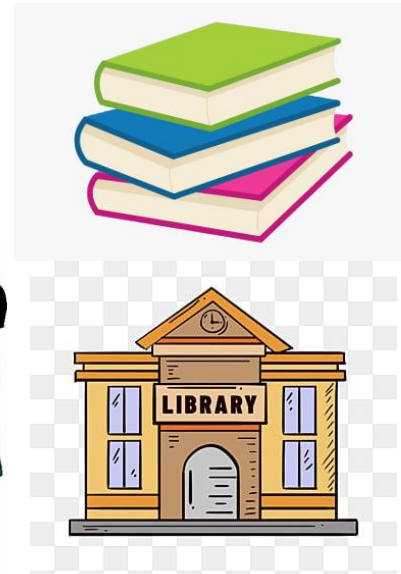
The tasks are different

Similarly

“+” with integer

“+” with complex number

The characteristics/processing are different.



C++: Overloading

In C++ the “Overloading” is the way to realize polymorphism.

Two types of overloading:

- Function/ Method overloading

- Operator overloading

C++ Syntax: Function Overloading

```
void print(int i) {  
    cout << " Here is int " << i << endl;  
}  
void print(double f) {  
    cout << " Here is float " << f << endl;  
}  
void print(char const *c) {  
    cout << " Here is char* " << c << endl;  
}  
  
int main() {  
    print(10);  
    print(10.10);  
    print("ten");  
    return 0;  
}
```

C++ Syntax : Operator Overloading

```
#include<iostream>
using namespace std;
class complex {
    int real, img;
public:
    complex() {
        real = 0; img = 0;
    }
    complex(int r, int i) {
        real = r; img = i;
    }
};
```

```
complex operator + (complex const &obj){
    complex res;
    res.real = real + obj.real;
    res.img = img + obj.img;
    return res;
}
```

```
void print() {
    cout << real << " + " << img << "i" << endl;
}

};
```

```
int main(){
    complex c1(10, 5), c2(2, 4);
    complex c3 = c1 + c2;
    c3.print();
    return 0;
}
```

Overview of C++ Polymorphism

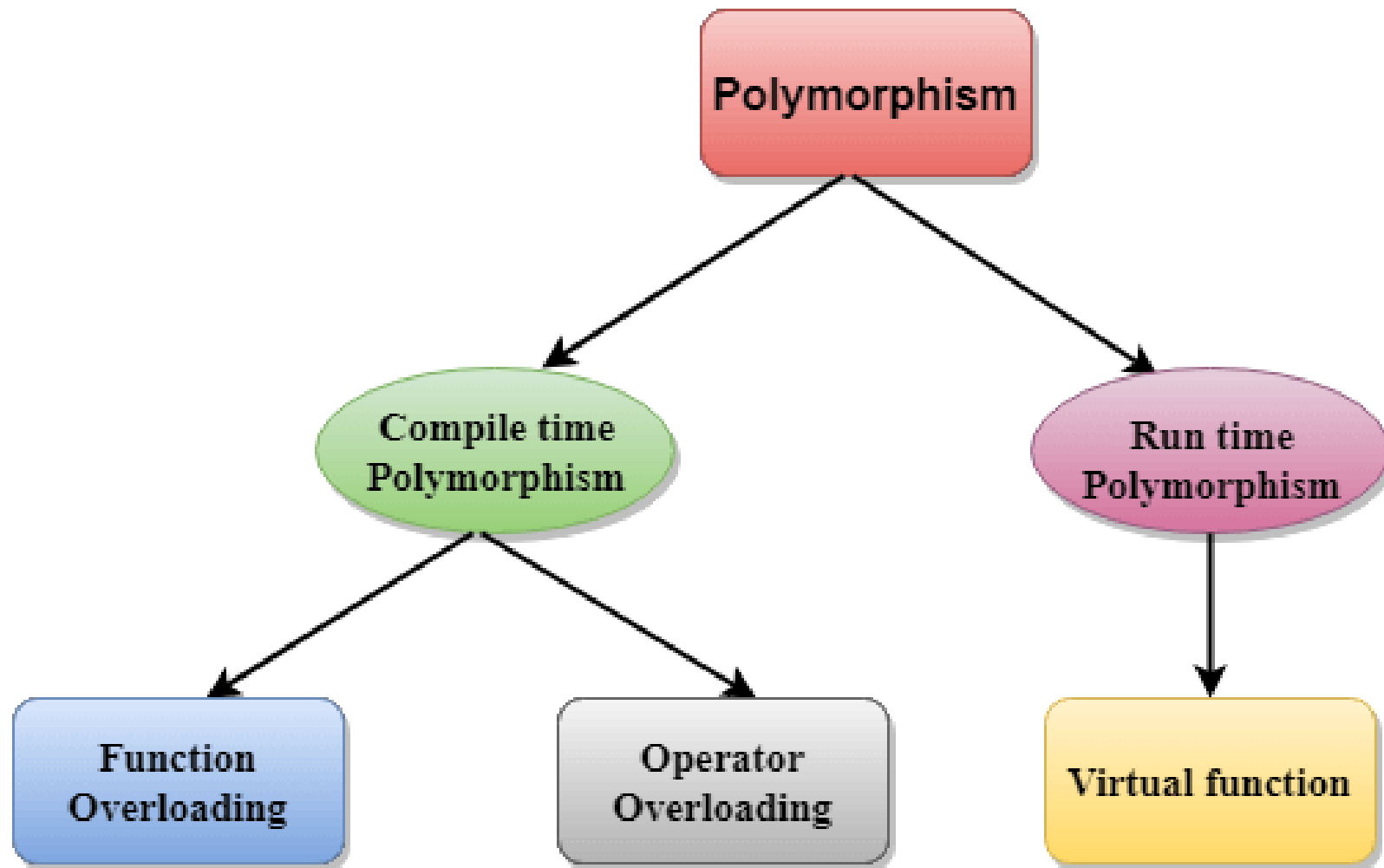
- The word **polymorphism** means having many forms.
- C++ polymorphism means that a call to a member function will cause a different function to be executed depending on the type of object that invokes the function.

Real Life Example Of Polymorphism

Let's consider a real-life example of polymorphism.

- A lady behaves like a teacher in a classroom, mother or daughter in a home and customer in a market.
- Here, a single person is behaving differently according to the situations.

Types of polymorphism in C++:



Compile time polymorphism:

- The overloaded functions are invoked by matching the type and number of arguments.
- This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time.
- It is achieved by **function overloading and operator overloading** which is also known as **static binding** or early binding.

Function Overloading:

- When there are multiple functions with same name but different parameters then these functions are said to be overloaded.
- Functions can be overloaded by change in number of arguments or/and change in type of arguments.
- The function is redefined by either using different types of arguments or a different number of arguments.
- It is only through these differences that a compiler can differentiate between functions.
- We can overload: methods, constructors, and indexed properties.

Advantages of function Overloading in C++

- The main advantage of function overloading is that it improves code readability and allows code reusability.
- The use of function overloading is to save memory space, consistency, and readability.
- It speeds up the execution of the program.
- Code maintenance also becomes easy.
- Function overloading brings flexibility to code.
- The function can perform different operations and hence it eliminates the use of different function names for the same kind of operations.

// C++ program for function overloading

#include <iostream>

using namespace std;

class Function1

{

public:

// function with 1 int parameter

void func(int x)

{

cout << "value of x is " << x << endl;

}

// function with same name but 1 double parameter

void func(double x)

{

cout << "value of x is " << x << endl;

}

// function with same name and 2 int parameters

void func(int x, int y)

{

cout << "value of x and y is " << x << ", " << y << endl;

}

};

```
int main() {  
    Function1 obj1;  
    // Which function is called will depend on the parameters passed  
  
    // The first 'func' is called  
    obj1.func(7);  
  
    // The second 'func' is called  
    obj1.func(9.132);  
    // The third 'func' is called  
  
    obj1.func(85,64);  
    return 0;  
}
```

```

#include <iostream>
using namespace std;
class Cal {
    public:
    static int add(int a,int b){
        return a + b;
    }
    static int add(int a, int b, int c)
    {
        return a + b + c;
    }
};
int main(void) {
    Cal C;
    cout<<C.add(10, 20)<<endl;
    cout<<C.add(12, 20, 23);
    return 0;
}

```

Program of function overloading when number of arguments vary.

OUTPUT
30
55

// class object declaration.

Program of function overloading with different types of arguments.

```
#include<iostream>
using namespace std;
int mul(int,int);
float mul(float,int);

int mul(int a,int b)
{
    return a*b;
}
float mul(double x, int y)
{
    return x*y;
}
int main()
{
    int r1 = mul(6,7);
    float r2 = mul(0.2,3);
    std::cout << "r1 is : " <<r1<< std::endl;
    std::cout <<"r2 is : " <<r2<< std::endl;
    return 0;
}
```

OUTPUT
r1 is : 42
R2 is : 0.6

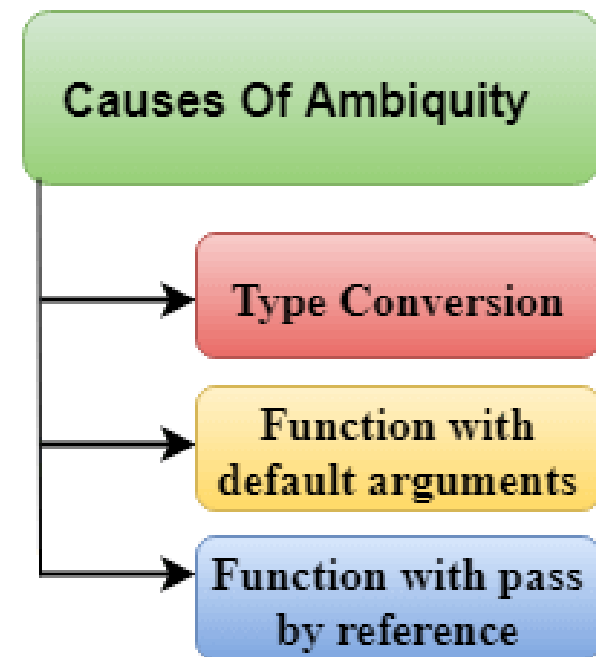
Disadvantages of function Overloading in C++

- Function declarations that differ only by its return type cannot be overloaded with function overloading process.
- Member function declarations with the same parameters or the same name types cannot be overloaded if any one of them is declared as a static member function.

```
class XYZ{  
    static void func();  
    void func(); // error  
};
```

Function Overloading and Ambiguity

- When the compiler is unable to decide which function it should invoke first among the overloaded functions, this situation is known as function overloading ambiguity.
- The compiler does not run the program if it shows ambiguity error.
- Causes of Function Overloading:
 - Type Conversion.
 - Function with default arguments.
 - Function with pass by reference.



Type Conversion:

```
#include<iostream>
using namespace std;
void fun(int);
void fun(float);
void fun(int i)
{
    std::cout << "Value of i is : " <<i<< std::endl;
}
void fun(float j)
{
    std::cout << "Value of j is : " <<j<< std::endl;
}
int main()
{
    fun(12);
    fun(1.2);
    return 0;
}
```

The example shows an error "**call of overloaded 'fun(double)' is ambiguous**". The fun(12) will call the first function. The fun(1.2) calls the second function according to our prediction. But this does not refer to any function as in C++, all the floating point constants are treated as double not as a float. If we replace float to double, the program works. Therefore, this is a type conversion from float to double.

Function with Default Arguments

```
#include<iostream>
using namespace std;
void fun(int);
void fun(int,int);
void fun(int i)
{
    std::cout << "Value of i is : " <<i<< std::endl;
}
void fun(int a,int b=9)
{
    std::cout << "Value of a is : " <<a<< std::endl
;
    std::cout << "Value of b is : " <<b<< std::endl
;
}
int main()
{
    fun(12);

    return 0;
}
```

The example shows an error "call of overloaded 'fun(int)' is ambiguous". The fun(int a, int b=9) can be called in two ways: first is by calling the function with one argument, i.e., fun(12) and another way is calling the function with two arguments, i.e., fun(4,5). The fun(int i) function is invoked with one argument. Therefore, the compiler could not be able to select among fun(int i) and fun(int a,int b=9).

Function with pass by reference

```
#include <iostream>
using namespace std;
void fun(int);
void fun(int &);
int main()
{
    int a=10;
    fun(a); // error, which f()?
    return 0;
}
void fun(int x)
{
    std::cout << "Value of x is : " <<x<< s
    td::endl;
}
void fun(int &b)
{
    std::cout << "Value of b is : " <<b<<
    std::endl;
}
```

The example shows an error **"call of overloaded 'fun(int&)' is ambiguous"**.

The first function takes one integer argument, and the second function takes a reference parameter as an argument. In this case, the compiler does not know which function is needed by the user as there is no syntactical difference between the `fun(int)` and `fun(int &)`.

Exercise

1. Create a class Rectangle with overloaded constructors:

Rectangle() → default constructor

Rectangle(int l, int w) → parameterized constructor

Rectangle(int s) → square constructor (length = width)

Create objects using all three constructors and print the area.

2. Write a program that overloads a function max() to find the **maximum value** among:

- two integers
- three integers
- two floating-point numbers

3. Explain whether the following overloading is valid or invalid:

void fun(int a);

void fun(int a = 5);



Operator Overloading

Operator Overloading

- In C++, it can add special features to the functionality and behavior of already existing operators like arithmetic and other operations
- This means C++ has the ability to provide the operators with a special meaning for a data type, this ability is known as operator overloading.
- For example, we can overload an operator '+' in a class like String so that we can concatenate two strings by just using +.
- Other example classes where arithmetic operators may be overloaded are Complex Number, Fractional Number, Big Integer, etc.

The advantage of Operators overloading is to perform different operations on the same operand.

Operator that cannot be overloaded are as follows:

- Scope operator (::)
- Sizeof
- member selector(.)
- member pointer selector(*)
- ternary operator(?:)

```
return_type class_name :: operator op(argument_list)
{
    // body of the function.
}
```

•Syntax of Operator Overloading

Where the **return type** is the type of value returned by the function.

class_name is the name of the class.

operator op is an operator function where op is the operator being overloaded, and the operator is the keyword.

Operator function must be either non-static (member function) or friend function.
Operator Overloading can be done by using **three approaches**, they are

- 1.Overloading unary operator.
- 2.Overloading binary operator.
- 3.Overloading binary operator using a friend function.

Implementing of Operator overloading:

1. Member function: It is in the scope of the class in which it is declared.
2. Friend function: It is a non-member function of a class with permission to access both private and protected members.

Difference between Member and friend function:

Member function:

- 1.The number of parameters to be passed is reduced by one, as the calling object is implicitly supplied is an operand.
- 2.Unary operators takes no explicit parameters.
- 3.Binary operators take only one explicit parameter.

Friend Function:

- 1.More number of parameters can be passed.
- 2.Unary operators take one explicit parameter.
- 3.Binary operators take two explicit parameters.

Rules for Operator Overloading

- Existing operators can only be overloaded, but the new operators cannot be overloaded.
- The overloaded operator contains atleast one operand of the user-defined data type.
- We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators. Example: = operator
- When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.
- When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

Overloading Unary Operator

Let us consider to overload (-) unary operator. In unary operator function, no arguments should be passed. It works only with one class objects. It is an overloading of an operator operating on a single operand.

Example:

Assume that class Distance takes two member object i.e. feet and inches, create a function by which Distance object should decrement the value of feet and inches by 1 (having single operand of Distance Type).

```
#include <iostream>
using namespace std;
class Distance {
public:
```

C++ program to show unary operator overloading

```
    int feet, inch;
    Distance(int f, int i)
    {
        this->feet = f;
        this->inch = i;
    }
    // Overloading(-) operator to perform decrement operation of Distance object
    void operator-()
    {
        feet--;
        inch--;
        cout << "\nFeet & Inches(Decrement): " << feet << " " << inch;
    }
};

int main()
{
    // Declare and Initialize the constructor
    Distance d1(8, 9);
    // Use (-) unary operator by single operand
    -d1;
    return 0;
}
```

Every object in C++ has access to its own address through an important pointer called **this** pointer. The **this** pointer is an implicit parameter to all member functions. Therefore, inside a member function, this may be used to refer to the invoking object.

Output: Feet & Inches(Decrement): 7'8

Overloading Binary Operator

In binary operator overloading function, there should be one argument to be passed. It is overloading of an operator operating on two operands.

```
#include <iostream>
using namespace std;
class A
{
    int x;
    public:
        A(){}
        A(int i)
        {
            x=i;
        }
        void operator+(A);
};
void A :: operator+(A a)
{
    int m = x+a.x;
    cout<<"The result of the addition of two objects is : "<<m;

}
```

```
int main()
{
    A a1(5);
    A a2(4);
    a1+a2;
    return 0;
}
```

OUTPUT:

The result of the addition of two objects is : 9

Overloading Binary Operator: Example

```
#include<iostream>
using namespace std;
class Complex {
private:
```

```
    int real, imag;
```

```
public:
```

```
    Complex(int r = 0, int i =0) {real = r; imag = i;}
```

```
    // This is automatically called when '+' is used with between two Complex objects
```

```
    Complex operator + (Complex const &obj) {
```

```
        Complex res;
```

```
        res.real = real + obj.real;
```

```
        res.imag = imag + obj.imag;
```

```
        return res;
```

```
    }
```

```
    void print() { cout << real << " + i" << imag << endl; }
```

```
};
```

```
int main()
```

```
{
```

```
    Complex c1(10, 5), c2(2, 4);
```

```
    Complex c3 = c1 + c2; // An example call to "operator+"
```

```
    c3.print();
```

```
}
```

Program to overload the + operator.

OUTPUT:

12+i9

Overloading Binary Operator: Example

```
#include <iostream>
using namespace std;
class Distance {
public:
    int feet, inch;
    Distance()
    {
        this->feet = 0;
        this->inch = 0;
    }
    // Constructor to initialize the object's value
    // Parameterized Constructor
    Distance(int f, int i)
    {
        this->feet = f;
        this->inch = i;
    }
}
```

// Overloading (+) operator to perform addition of two distance object

Distance operator+(Distance& d2) // Call by reference

```
{    // Create an object to return
    Distance d3;
    // Perform addition of feet and inches
    d3.feet = this->feet + d2.feet;
    d3.inch = this->inch + d2.inch;
    // Return the resulting object
    return d3;
}
```

};

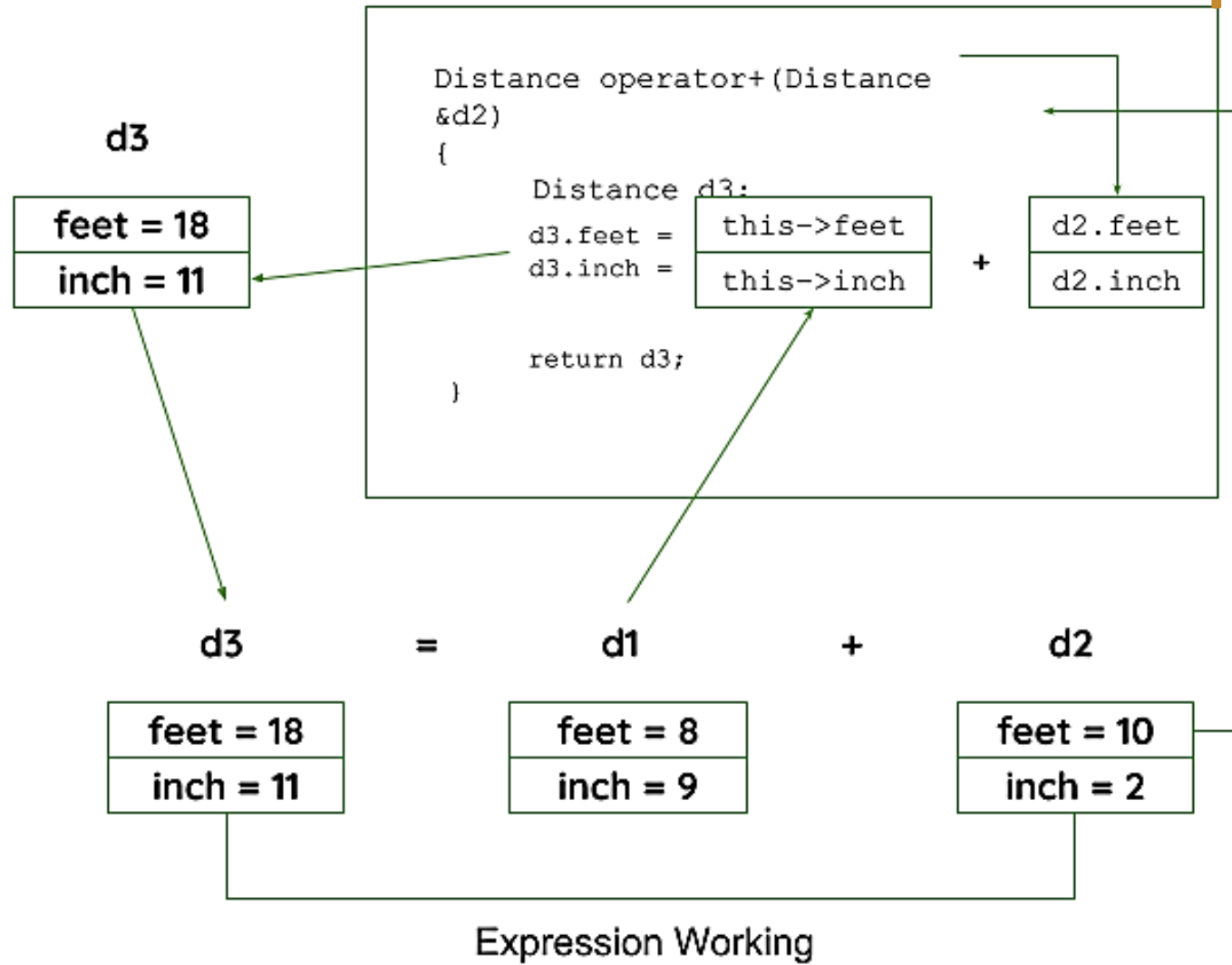
int main()

```
{    // Declaring and Initializing first object
    Distance d1(8, 9);
    // Declaring and Initializing second object
    Distance d2(10, 2);
    // Declaring third object
    Distance d3;
    // Use overloaded operator
    d3 = d1 + d2;
    // Display the result
    cout << "\nTotal Feet & Inches: " << d3.feet << " " << d3.inch;
    return 0;
}
```

Example cont..

Output: Total Feet & Inches: 18'11

Example cont..



Overloading Binary Operator using a Friend function:

- In this approach, the operator overloading function must precede with friend keyword, and declared inside the class scope.
- A friend operator function takes two parameters when overloading a binary operator, and one parameter when overloading a unary operator.
- All the working and implementation would same as binary operator function except this function will be implemented outside of the class scope.

```
#include <iostream>
using namespace std;
class Distance {
public:
```

// C++ program to show binary operator overloading

```
    int feet, inch;
```

```
    Distance()
```

```
{
```

```
        this->feet = 0;
```

```
        this->inch = 0;
```

```
}
```

```
// Constructor to initialize the object's value// Parameterized Constructor
```

```
Distance(int f, int i)
```

```
{
```

```
        this->feet = f;
```

```
        this->inch = i;
```

```
}
```

```
// Declaring friend function using friend keyword
```

```
friend Distance operator+(Distance&, Distance&);
```

```
};
```

```
// Implementing friend function with two parameters
Distance operator+(Distance& d1, Distance& d2) // Call by reference
{
    // Create an object to return
    Distance d3;
    // Perform addition of feet and inches
    d3.feet = d1.feet + d2.feet;
    d3.inch = d1.inch + d2.inch;
    // Return the resulting object
    return d3;
}

int main()
{
    // Declaring and Initializing first object
    Distance d1(8, 9);
    // Declaring and Initializing second object
    Distance d2(10, 2);
    // Declaring third object
    Distance d3;
    // Use overloaded operator
    d3 = d1 + d2;
    // Display the result
    cout << "\nTotal Feet & Inches: " << d3.feet << " " << d3.inch;
    return 0;
}
```

Total Feet & Inches: 18'11

Disadvantages of an operator overloading:

In operator overloading, any C++ existing operations can be overloaded, but some exceptions.

Which operators Cannot be overloaded?

1. Conditional [?:], size of, scope(::), Member selector(.), member pointer selector(*) and the casting operators.
2. We can only overload the operators that exist and cannot create new operators or rename existing operators.
3. At least one of the operands in overloaded operators must be user-defined, which means we cannot overload the minus operator to work with one integer and one double. However, you could overload the minus operator to work with an integer and a mystring.
4. It is not possible to change the number of operands of an operator supports.
5. All operators keep their default precedence and associations (what they use for), which cannot be changed.
6. Only built-in operators can be overloaded.

Exercises

1. Design a class Complex with attributes real and imag for real and imaginary parts. Overload the binary * operator to perform multiplication of two complex numbers. Write a program to multiply two complex objects and display the result.
2. Design a class Counter that stores an integer value. Overload the unary ++ operator to increment the value of the counter by 1. Write a program to demonstrate prefix increment using this operator.
3. Design a class Employee with attributes name and salary. Overload the relational < operator to compare the salaries of two employees. Write a program to determine which employee has a lower salary.

Thank You



Data Conversion

Data Conversion

Conversion between basic types and objects: int to class type

```
#include <iostream>
using namespace std;

class Box {
    int length;
public:
    // Conversion constructor
    Box(int l) {
        length = l;
    }

    void show() {
        cout << "Length = " << length << endl;
    }
};

int main() {
    Box b1 = 10; // implicit conversion: int → Box
    b1.show();

    Box b2(25); // explicit creation
    b2.show();

    return 0;
}
```

Class to int type

```
#include <iostream>
using namespace std;
class Box {
    int length;
public:
    Box(int l) : length(l) {}
    // Conversion function: class → int
    operator int() {
        return length;
    }
};

int main() {
    Box b1(10);
    int x = b1; // automatic conversion: calls operator int()
    cout << "Length = " << x << endl;
    return 0;
}
```